

PuzzleForge

An Agentic Puzzle Game Design and Generation System

Final Report

94-815 Agentic Technologies

Team: Yixiao Li, Kaizhen Tan, Hanzhe Hong

Track: A -- Technical Build

1. Problem Statement

Designing solvable puzzle games is difficult because it requires balancing creativity (interesting level layouts) with correctness (every level must be solvable and avoid deadlocks). A single large language model cannot reliably verify solvability; it can only guess. This creates a fundamental challenge: the creative generation task and the verification task require different capabilities, and conflating them inside one model leads to puzzles that appear plausible but are broken.

PuzzleForge addresses this problem by separating creative design from automated verification across five coordinated agents in a LangGraph pipeline. The system transforms a user's free-text game concept into a playable, browser-based Sokoban puzzle game where every shipped level is provably solvable by a breadth-first search (BFS) solver.

2. Architecture

2.1 System Overview

PuzzleForge is organized as a two-layer architecture with five agents. The Design Layer contains the two creative agents: the Game Designer (LLM, temperature 0.8), which generates the overall game specification from a user concept, and the Level Designer (LLM, temperature 0.7), which creates and redesigns individual grid layouts. The Implementation Layer contains three agents responsible for verification and coordination: the Developer / Orchestrator, which translates designs into a structured configuration and routes failures using deterministic rules; the QA Tester, which uses a BFS solver to verify solvability, difficulty, and diversity (zero tokens, no LLM); and the Debugger (LLM, temperature 0.3), which applies minimal tile-level fixes to broken configurations.

The guiding design principle is deterministic where possible, LLM where necessary. Solvability verification, routing classification, and iteration control are handled by code, not LLM judgment. LLMs are reserved for the creative work they do best — theme design, narrative framing, and spatial arrangement — while correctness guarantees are enforced by deterministic components.

Agent	Layer	Role	Tools	LLM Tokens
Game Designer	Design	Generate game spec from user concept	LLM (temp 0.8)	Yes
Level Designer	Design	Create/redesign grid layouts	LLM (temp 0.7)	Yes
Developer/Orchestrator	Implementation	Translate designs, route failures	Template engine, routing rules	No (0 tokens)
QA Tester	Implementation	Verify solvability, difficulty, diversity	BFS solver	No (0 tokens)
Debugger	Implementation	Minimal config fixes	LLM (temp 0.3)	Yes

2.2 Pipeline Flow

The pipeline runs in a forward pass followed by a conditional feedback loop. A user's concept flows through the Game Designer, the Level Designer, and the Developer (which translates the designs) before

reaching the QA Tester, which verifies every level with the BFS solver. If every level passes, the system finalizes and produces the playable game. If any level fails, the Developer classifies each failure and routes it either to the Debugger for a small configuration patch or back to the Level Designer for a full redesign. The fixed levels are then re-tested by QA. The loop terminates once all levels pass, the three-cycle cap is reached, the token budget is exhausted, or the five-minute wall-clock timeout fires — whichever comes first.

```
User Input --> Game Designer --> Level Designer --> Developer (translate)
--> QA Tester (BFS solver) --> [route_after_qa]:
    all_pass      --> Finalize (playable game)
    has_failures --> Developer (classify & route):
        config_bugs --> Debugger --> Apply Patches --> QA re-test
        design_flaws --> Level Designer (redesign) --> Developer --> QA re-
test
    max_cycles    --> Finalize (with failures logged)
    budget/timeout --> Finalize (best available)
```

2.3 Orchestration Framework

The pipeline is implemented on top of LangGraph's StateGraph. A shared TypedDict holds pipeline state across all nodes; append-only fields such as the trace log and accumulated debug patches use `Annotated[List, operator.add]` so history is preserved without manual bookkeeping. The graph consists of nine compiled nodes connected by four conditional edges that encode the routing logic as explicit graph structure rather than hidden prompt instructions. This makes the control flow auditable: any path through the pipeline can be reproduced from the trace.

2.4 Routing Criteria

When QA reports failures, the Developer classifies each failure using ten deterministic rules implemented in `src/orchestrator/routing.py`. No LLM judgment is involved at this step. Five criteria (C1–C5) identify configuration bugs — placement issues, count mismatches, player on walls — and route the level to the Debugger. Five more criteria (D1–D5) identify design flaws — grids too small, solver timeouts, unresolved deadlocks, diversity violations — and route the level back to the Level Designer.

ID	Criterion	Classification	Routes To
C1-C5	Config bugs (placement issues, count mismatches)	Config bug	Debugger
D1-D5	Design flaws (grid too small, solver timeout, diversity)	Design flaw	Level Designer

2.5 Stop Conditions

The pipeline terminates when any of four conditions is satisfied: all levels pass QA, three debug cycles have been exhausted, the 80K token budget has been exceeded, or the five-minute wall-clock timeout has fired.

3. Implementation Details

3.1 BFS Sokoban Solver

The ground-truth verification tool lives in `src/solver/sokoban_solver.py`. It runs a breadth-first search over states represented as `(player_position, frozenset(box_positions))`, which guarantees that any solution it returns is a minimum-move solution. Three deadlock detectors — corner, wall-line, and 2x2 freeze — prune clearly unsolvable branches so the search terminates promptly on broken levels, and a 200,000-state cap prevents runaway computation on pathological inputs. Beyond solvability, the module exposes `estimate_difficulty()` (a one-to-five rating derived from solution length, states explored, and grid area) and `level_similarity()` (a Jaccard index used for diversity checks).

3.2 Inter-Agent Communication

All inter-agent communication passes through Pydantic v2 models. Invalid JSON triggers an automatic retry in which the validation error is fed back into the next prompt so the LLM has specific guidance on what to fix. The key schemas are `GameSpec`, `LevelDefinition`, `QAReport`, `DebugPatch`, and `RoutingDecision`; every handoff between agents is validated at the boundary, so a malformed payload is caught immediately instead of propagating silently.

3.3 Game Rendering

The HTML/JS template at `src/engine/templates/sokoban.html` renders playable games in the browser. It supports both push and slide mechanics — push moves a box exactly one tile in the direction of movement, while slide sends the box sliding until it collides with a wall or another box. Five color themes (dark, earthy, icy, gothic, space) are driven by the `theme_details` block in the `GameConfig`, giving each generated game a distinct visual identity. Entities are rendered as emoji chosen by the Game Designer (players, boxes, targets, and optional wall glyphs), and the game header displays a mechanic badge so players understand the movement rule at a glance. The player interacts through keyboard or touch controls, with undo and per-level progression available throughout.

Two recent additions make the generated games more challenging. First, move-limit enforcement: each level now carries a `min_moves` value computed by the BFS solver (the shortest possible solution). The UI displays `Moves: X / limit` and a `Best: N par` indicator, where the limit is `min_moves + 2`. If the player exceeds the limit without solving the level, the game transitions to a `lost` state and prompts a reset. The value of `min_moves` flows from the QA Tester's report into `game_config.levels` during the `finalize` node, so the limit is baked into the rendered HTML. Second, the Game Designer and Level Designer prompts were tightened to require a meaningful challenge curve: a minimum grid of 6x6, a two-box tutorial scaling to four- or five-box expert levels, and layout-driven progression through interior walls and multi-room structures rather than trivial open rooms.

4. Evaluation Results

4.1 Live Pipeline Runs

We exercised the pipeline on four concepts covering both mechanics and both dominant theme families. Across these runs, the pipeline completed successfully in every case and produced a playable, solver-verified game on the first pass for the gothic-dungeon run; the remaining three runs consumed their full three-cycle debug budget to repair individual failing levels. The table below summarizes the per-run outcomes.

Run	Concept	Mechanic	Theme	Levels	Solvable	Rate	Cycles	Tokens	Time	Status
RUN-01	Space station cargo robot	push	space	5	4/5	80%	3	6,093	28s	completed_with_failures
RUN-02	Penguin sliding ice blocks	slide	icy	4	3/4	75%	3	8,538	44s	completed_with_failures
RUN-03	Gothic dungeon gem puzzle	push	gothic	5	5/5	100%	0	2,969	15s	completed
RUN-04	Zero-gravity curling	slide	space	3	2/3	67%	3	8,193	36s	completed_with_failures

Aggregated across all runs, the overall solvability rate was 82% (14 of 17 levels), with a meaningful gap between mechanics: push solvability reached 90% (9 of 10) while slide solvability reached 71% (5 of 7). Every pipeline run completed cleanly, for a 100% completion rate. Mean token usage was 6,448 per run — comfortably inside the 80K budget — and mean wall-clock time was 30.9 seconds. Debug cycles per run ranged from zero to three, and the Game Designer correctly chose the mechanic from the concept in all four cases (push for warehouse and dungeon themes, slide for ice and curling themes).

4.2 Success Criteria Assessment

Criterion	Target	Actual	Status
Solvability rate (push)	>= 90%	90% (9/10)	PASS
Solvability rate (slide)	>= 90%	71% (5/7)	NEEDS IMPROVEMENT
Solvability rate (overall)	>= 90%	82% (14/17)	PARTIAL
Pipeline completion	>= 85%	100% (4/4)	PASS
Latency	<= 4 minutes	Mean 30.9s	PASS
Token efficiency	<= 80K per run	Mean 6,448	PASS
Routing accuracy	>= 75%	100% (all failures correctly routed)	PASS
Mechanic selection	Game Designer matches concept	4/4 correct	PASS

Theme diversity	Each run has distinct visual identity	4 unique themes	PASS
Difficulty curve	$\geq 80\%$ monotonic	4/4 runs monotonic (100%)	PASS

Slide-mechanic levels are harder for the LLM to design correctly than push levels because the Level Designer must reason about sliding physics — boxes continue until they hit a wall or another box, so placement decisions have longer-range consequences. Push levels reach the 90% target while slide levels trail at 71%. The slide-specific guidance added to the Level Designer prompt partially mitigates this gap, but further improvement is needed (see F-007 in the failure log).

4.3 Baseline Comparison

To demonstrate the value of the multi-agent architecture, we ran a single-LLM baseline: one GPT-4o call generates all five levels, with no solver verification, no debug loop, and no specialized agents. We then verified the baseline outputs with the same BFS solver used inside the pipeline.

Metric	Single-LLM Baseline	Multi-Agent Pipeline	Improvement
Push solvability	100% (10/10)	90% (9/10)	comparable
Slide solvability	30% (3/10)	71% (5/7)	+41 pp
Overall solvability	65% (13/20)	82% (14/17)	+17 pp
Verified solvable	No (LLM self-report)	Yes (BFS solver)	critical
Tokens per run	1,348	6,448	4.8x (cost of verification)

The comparison produces four findings. First, slide mechanics expose a clear limit on LLM spatial reasoning: the baseline manages only 30% solvability on slide levels because the model cannot reliably reason about sliding physics, while the pipeline's debug feedback loop more than doubles that to 71%. Second, push levels are relatively easy for LLMs, and both approaches achieve high solvability there — but the pipeline's contribution for push is not higher accuracy so much as verified correctness, since baseline levels are unverified and could silently ship broken puzzles. Third, the debug loop is the key differentiator: the pipeline spends roughly five times more tokens per run, and this extra spend buys solver verification and iterative repair, which for slide levels is the difference between 30% and 71%. Fourth, the baseline has no way to know which of its levels are broken; the pipeline, by contrast, catches every unsolvable level and either repairs it or logs it for downstream attention.

4.4 Test Cases Summary

We executed eleven scenario-level test cases, recorded in `eval/test_cases.csv`. All eleven passed, covering end-to-end happy paths (TC-01, TC-05), solvability and routing behavior (TC-02, TC-04, TC-09, TC-10), quality checks around difficulty and diversity (TC-03, TC-06, TC-07), and validation and edge cases including the slide-mechanic selection path (TC-08, TC-12). In addition to the scenario tests, fifteen unit-level tests pass: five against the solver and ten against the routing criteria.

5. Failure Analysis

5.1 Failures Discovered and Fixed

We discovered and resolved five issues during live Phase 3 testing. The three most instructive are summarized below; all eight documented failures appear in `eval/failure_log.md` (F-001 through F-008).

F-004: Silent Redesign Failure (Severity: HIGH)

The Level Designer's `_redesign_levels()` function originally caught all exceptions with `except Exception: new_levels = []`. When the LLM returned invalid JSON during a redesign pass, the output was silently discarded. The broken level was never replaced, so the pipeline ran all three debug cycles without making progress. The root cause was overly broad exception handling that prioritized graceful degradation over correctness: the silent failure meant the feedback loop was fundamentally broken — QA kept reporting the same failures, the Developer kept routing them, but the Level Designer's fixes were thrown away before the Developer could apply them. The fix was to replace the blanket `except` with a narrow retry that re-invokes the LLM using the specific validation error as feedback, and to extend the redesign prompt with the original level's dimensions and solver-timeout guidance. The governance lesson is that silent failure in a feedback loop is worse than a crash. The system appeared to be working (three cycles completed) while doing nothing; explicit error handling and loud observability are essential for multi-agent reliability.

F-005: Diversity Hard-Blocking (Severity: HIGH)

When all five levels were solvable but one pair happened to exceed a Jaccard similarity threshold of 0.5, the pipeline treated the diversity violation as a hard failure and burned all three debug cycles trying to fix a cosmetic issue. The Debugger cannot fix diversity — it only adjusts tile positions within a single level — so those cycles were simply wasted. The routing system correctly classified diversity violations as design flaws (criterion D5) and sent them to the Level Designer; the real bug was upstream, in `route_after_qa()`, which treated "has failures" as a single condition regardless of severity. Solvability failures and cosmetic violations were consuming the same retry budget. The fix introduces a soft-fail path: if every level is already solvable and at least one debug cycle has been attempted, the pipeline accepts the result and finalizes. Diversity is now treated as a soft constraint. The lesson is that not all failures are equal; a loop that treats cosmetic issues with the same urgency as critical bugs will waste its iteration budget on the wrong problems.

F-006: Debugger Processing Stale Levels (Severity: MEDIUM)

The Debugger originally computed `levels_to_fix` by unioning `config_bug_ids` (from the append-only `routing_decisions` list) with `failed_level_ids`. Because `routing_decisions` accumulates across all cycles via `operator.add`, levels that had been repaired in cycle 1 reappeared in cycles 2 and 3. The root cause is a mismatch between state pattern and consumption pattern: the `operator.add` reducer that works well for trace logging, where complete history is valuable, is dangerous for routing decisions, where only the current cycle should drive behavior. The fix changes the Debugger to read only `failed_level_ids`, which is reset each cycle by `developer_route_node`.

The governance lesson is that append-only state is excellent for observability but hazardous for decision-making; agents that act on state must clearly distinguish historical records from current instructions.

5.2 Failure Patterns

Across all eight documented failures, three patterns emerge. First, silent failures are the most dangerous class of bug: F-003 (Pydantic retry) and F-004 (redesign retry) both involved invalid LLM output, but F-003 was caught explicitly by Pydantic and recovered through a transparent retry, while F-004 was silently swallowed. The difference is explicit validation versus broad exception handling. Second, feedback loops need severity awareness: F-005 showed that a loop without tiered severity will treat every issue equally and can spend its entire retry budget on low-priority problems while critical bugs remain. Third, append-only state requires careful consumption: F-006 showed that the same state pattern that enables strong observability can produce incorrect behavior when agents read accumulated history as if it were current instructions.

6. Governance Reflection

6.1 Governance Controls and Their Effectiveness

Control	Implementation	Effectiveness	Evidence
BFS solver as ground truth	QA Tester uses deterministic solver (push + slide), not LLM	Highly effective	14/17 levels verified solvable across 4 runs (push: 9/10, slide: 5/7)
Deterministic routing	10 formal criteria in <code>routing.py</code>	Effective	All routing decisions correct; no misclassification observed
Debugger bounded authority	25% tile change threshold; temp 0.3	Effective	Escalation correctly triggered when needed
3-cycle iteration cap	Hard limit in <code>route_after_qa()</code>	Effective but blunt	Prevents infinite loops but doesn't differentiate failure severity
80K token budget	Checked at conditional edges and in <code>call_llm()</code>	Effective	Never hit in practice (max 6,217 tokens per run)
Pydantic validation	All inter-agent JSON validated	Effective	Caught invalid outputs; retry mechanism works
Append-only trace logging	<code>Annotated[List, operator.add]</code>	Effective for observability	Full history preserved; but required careful consumption (F-006)

6.2 What We Would Change

With the benefit of hindsight, three changes stand out. The three-cycle cap should become severity-aware, allocating cycles based on failure type: solvability failures should take priority and diversity issues should get at most one attempt. Exception handling should be structured rather than broad; F-004 demonstrated that an `except Exception` inside a feedback loop can silently break the entire system, and every exception handler should either retry with feedback, log explicitly, or escalate rather than silently

continue. Finally, append-only state should be kept separate from decision state: `routing_decisions` is valuable as a permanent audit trail, but the Debugger should consume a fresh `current_cycle_bugs` field that is reset each cycle so there is no ambiguity about what the agent is supposed to act on.

6.3 Human-in-the-Loop

The current system is fully automated from concept to finished game. The human touches three points: the user provides the initial concept, receives the completed game and trace log, and can inspect every agent's decision through the Streamlit dashboard (which also allows adjusting settings and rerunning). For a production deployment we would add three additional oversight hooks: a routing override that lets a human reclassify a failure, a level editor that allows manual adjustments to generated layouts, and approval gates that require human sign-off before finalizing.

7. Lessons Learned

7.1 Technical Lessons

LangGraph's reducer pattern is powerful but requires discipline. `Annotated[List, operator.add]` elegantly handles trace accumulation, but any agent that acts on accumulated state must distinguish history from current instructions. Retry logic is essential for LLM-based agents: all three LLM agents (Game Designer, Level Designer, Debugger) can produce invalid JSON, and the retry-with-error-feedback pattern is both cheap (roughly one extra API call) and effective. The BFS solver is the system's most reliable component; it never produces false positives or false negatives, and grounding agent output in deterministic verification has turned out to be the single strongest governance mechanism in the system. Finally, soft and hard constraints must be made explicit: the diversity issue (F-005) showed that conflating "must fix" with "nice to fix" can exhaust the iteration budget on the wrong problems.

7.2 Process Lessons

Live testing reveals bugs that unit tests miss — all three Phase 3 bugs (F-004, F-005, F-006) passed unit tests but failed during live pipeline execution, because the interaction between agents under real conditions exposed issues that isolated testing could not. The feedback loop is the hardest part of the architecture to get right: the forward path from user input to QA worked correctly from Phase 2 onward, and every Phase 3 bug was in the feedback loop (redesign, debug, re-test). Observability pays for itself; the trace log made it possible to diagnose F-004 (silent redesign failure) by showing that the Level Designer was being invoked but producing no changes, and without the trace the problem would have been very difficult to find.

8. Individual Contribution Reflection

Yixiao Li

My responsibilities were the Game Designer agent, the Level Designer agent, and the BFS solver. During Phase 3 I discovered and fixed F-004 (silent redesign failure) by introducing a narrow retry with error feedback and improving the redesign prompt, F-005 (diversity hard-blocking) by adding a soft-fail path to

the graph, and F-006 (stale level processing) by correcting the Debugger's state consumption. I ran the four live pipeline tests, collected evaluation data, and wrote the failure analysis and evaluation sections. I also kept `test_cases.csv`, `evaluation_results.csv`, `failure_log.md`, and `version_notes.md` in sync with the code changes. The most important thing I learned is that multi-agent systems fail in the interactions between agents, not inside any single agent. Each agent worked correctly in isolation, but the way they shared state and responded to each other's outputs produced emergent bugs. The append-only state issue behind F-006 was especially eye-opening: a design pattern that is perfect for logging caused incorrect behavior when used for decision-making, and the lesson generalizes — boundaries between agents need explicit contracts, not just working code.

9. Evidence Package Summary

The evaluation and failure packages are shipped alongside the code. We executed eleven scenario-level test cases (see `eval/test_cases.csv`): two happy-path runs (TC-01, TC-05), a solvability-loop deadlock fix (TC-02), difficulty-curve detection (TC-03), design-flaw routing (TC-04), soft diversity (TC-06), layout-similarity metric (TC-07), Pydantic validation (TC-08), Debugger escalation (TC-09), solver-timeout routing (TC-10), and the slide-mechanic live run (TC-12); all eleven passed. Eight failure cases are documented in `eval/failure_log.md`, covering corner deadlocks, similar layouts, invalid mechanics, silent redesign, diversity hard-blocking, stale-level processing, and the two slide-solvability limitations. Pipeline run data, including the baseline comparison, lives in `eval/evaluation_results.csv`: four live API runs (two push, two slide) plus one sample-config verification, with an 82% solvability rate across 17 levels (push 90%, slide 71%), 100% completion, and correct mechanic selection in every run.

10. Future Improvements

Six improvements would meaningfully strengthen the system. First, a severity-aware iteration budget would allocate debug cycles based on failure severity rather than treating all failures equally, so solvability failures get priority and diversity issues get at most one attempt. Second, difficulty-curve enforcement during redesign would feed the full level set's difficulty ratings into the redesign prompt, helping maintain monotonic progression. Third, additional puzzle mechanics (switches, teleports) would extend the solver and template beyond push and slide. Fourth, human-in-the-loop routing overrides would let users reclassify a failure from the Streamlit dashboard before the next debug cycle. Fifth, parallel level generation would replace the current single-call strategy, reducing the impact of a single invalid level on the batch. Sixth, solver-guided level design would feed solver metrics (states explored, deadlock locations) back to the Level Designer during initial generation, not only during redesign, reducing the number of levels that fail QA on the first pass.

Appendix: Repository Structure

```
PuzzleForge/
+-- README.md           # Project overview and setup
+-- AI_USAGE.md         # AI tool usage disclosure
+-- requirements.txt     # Python dependencies
```

```

+-- .env.example           # Environment template
+-- docs/
|   +-- final_report.pdf   # Phase 3 final report (PDF)
|   +-- phase3_report.docx # Phase 3 final report (Word)
|   +-- architecture_diagram.pdf # System architecture diagram
|   +-- project_summary.pdf # One-page project summary
|   \-- screenshots/      # Screenshots with index
+-- media/
|   \-- demo_video_link.txt # 5-minute video link
+-- src/
|   +-- models.py          # Pydantic data models
|   +-- state.py           # LangGraph state schema
|   +-- config.py          # Configuration and thresholds
|   +-- main.py            # CLI entry point
|   +-- app.py             # Streamlit dashboard
|   +-- agents/            # 5 agent implementations
|   +-- solver/            # BFS Sokoban solver
|   +-- engine/            # HTML/JS game template engine
|   +-- orchestrator/     # LangGraph pipeline + routing
|   \-- utils/            # Trace logger, LLM wrapper
+-- eval/
|   +-- test_cases.csv     # 11 test scenarios with results
|   +-- evaluation_results.csv # Pipeline + baseline run data
|   +-- evaluation_plan.md # Evaluation methodology
|   +-- failure_log.md     # 8 documented failures
|   +-- baseline.py        # Single-LLM baseline script
|   +-- baseline_results.json # Baseline run results
|   \-- version_notes.md   # Version history
+-- outputs/
|   +-- game.html          # Latest generated game
|   +-- design_doc.json    # Latest design document
|   +-- trace.json         # Latest execution trace
|   \-- sample_traces/    # Sample configs for demo
\-- tools/
    +-- build_report_docx.py # Report Word builder
    \-- build_report_pdf.py  # Report PDF builder

```